Informe técnico
Modernización del Banco XYZ

Desarrollo Backend Avanzado: Spring Cloud y Batch

Migración batch · Backend for Frontend · Microservicios resilientes en la nube

Estudiante	Emily Rupay
Docente	_____
Sección	_____
Repositorio	github.com/EmilyRupay/Banco-xyz-cloud
Fecha	Octubre de 2026

Índice

1. Introducción

2. Contexto y problema

3. Objetivos y alcance

4. Arquitectura de la solución

5. Casos de uso

6. Parte 1 – Migración de procesos batch con Spring Batch

7. Parte 2 – Patrón Backend for Frontend (BFF)

8. Parte 3 – Microservicios resilientes con Spring Cloud

9. Integración entre componentes

10. Despliegue: contenedores y nube (AWS)

11. Pruebas y validación

12. Resultados y comparación con el sistema legacy

13. Desafíos enfrentados y soluciones

14. Propuestas de mejora y próximos pasos

15. Conclusiones

16. Referencias

1. Introducción

El presente informe documenta el diseño, la implementación y la validación de la solución backend desarrollada para la modernización tecnológica del **Banco XYZ**, en el marco de la Evaluación Final Transversal de la asignatura Desarrollo Backend III. El trabajo abarca la migración de procesos batch a **Spring Batch**, la implementación del patrón **Backend for Frontend** para los canales web, móvil y cajeros automáticos, y la construcción de **microservicios seguros y resilientes** con Spring Cloud, desplegables en contenedores y preparados para AWS.

Para cada componente se describe qué hace, cómo se integra con el resto y qué decisiones técnicas se tomaron, apoyándose en diagramas de arquitectura, casos de uso, componentes, secuencia y despliegue. El código fuente completo, las instrucciones de ejecución y la guía de despliegue se encuentran en el repositorio `github.com/EmilyRupay/Banco-xyz-cloud` (archivos `readme.md`, `instrucciones.md` y `despliegue.md`).

2. Contexto y problema

El Banco XYZ opera hace más de 30 años sobre una plataforma **legacy en mainframe basada en COBOL y scripts Shell** que soporta procesos críticos: procesamiento de transacciones, estados de cuenta, cálculo de intereses y gestión de clientes. Esta plataforma presenta:

- **Escalabilidad limitada**: el monolito escala completo o no escala; no es posible asignar más capacidad solo a pagos.
- **Altos costos de mantenimiento** y dificultad para integrar nuevas tecnologías y canales digitales.
- **Baja tolerancia a fallos**: un error en un módulo afecta a todo el sistema; los procesos batch fallidos se reejecutan manualmente.
- **Seguridad centralizada**, insuficiente para un entorno distribuido con múltiples canales.
- **Frontends acoplados**: web, móvil y cajeros reciben los mismos datos del backend monolítico, con sobrecarga innecesaria.

Los datos de entrada de los procesos batch provienen del repositorio `fin_legacy_data`, cuyos archivos CSV contienen errores intencionales típicos del legacy: montos negativos o en cero, campos vacíos, cuatro formatos de fecha, tipos de transacción inválidos, registros duplicados y edades fuera de rango.

3. Objetivos y alcance

Objetivo	Alcance implementado
Migrar los procesos batch legacy	3 jobs de Spring Batch con lectura, proceso y escritura; reintentos, omisión, particionamiento, multihilo, reinicio automático y verificación de equivalencia
Optimizar la experiencia por canal	3 BFF independientes (web, móvil, ATM) con autenticación/autorización propia y respuestas adaptadas
Desacoplar el monolito	Microservicios de Clientes, Cuentas y Pagos + notificaciones, cada uno con su base de datos
Resiliencia y seguridad	Resilience4j (Retry, CircuitBreaker, Bulkhead, fallbacks), OAuth2/JWT, HTTPS, defensa en profundidad
Tiempo real	Apache Kafka con 3 tópicos de eventos, productores, consumidor idempotente y Dead Letter Topics
Operación en la nube	Docker Compose (14 contenedores), monitoreo (Prometheus, Grafana, Zipkin), guía y scripts para AWS

4. Arquitectura de la solución

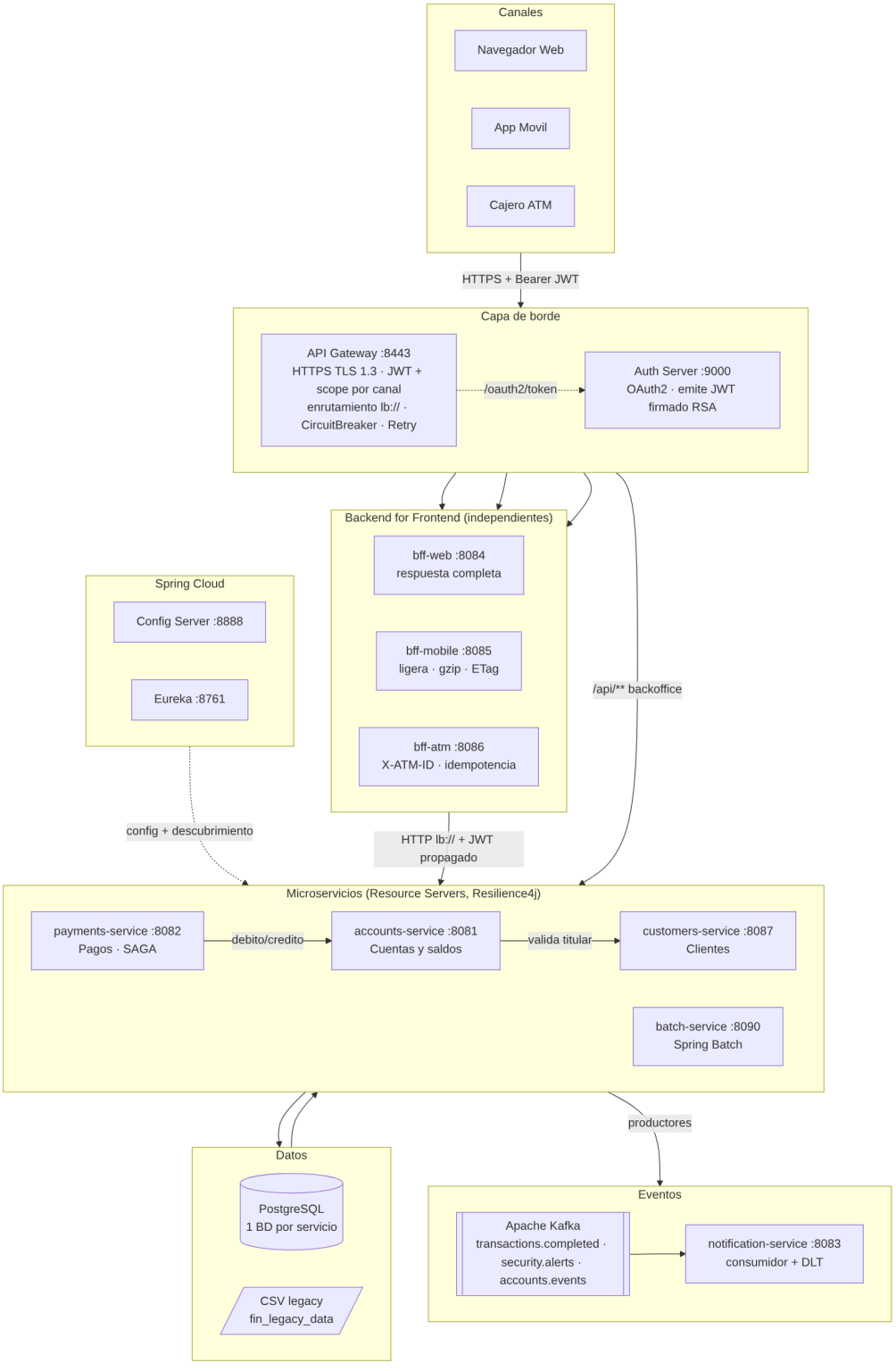


Figura 1. Diagrama de arquitectura general

La arquitectura se organiza en capas. Los **canales** se comunican exclusivamente con el **API Gateway** por HTTPS; el Gateway valida el JWT y el *scope* del canal, y enruta con balanceo de carga (`lb://`) hacia el **BFF** correspondiente. Los BFF agregan y adaptan datos de los **microservicios de negocio**, que son *Resource Servers* OAuth2 independientes con su propia base de datos. Los cambios de estado relevantes se publican como **eventos en Kafka**

y son consumidos por el servicio de notificaciones. La configuración es centralizada (**Config Server**) y la ubicación de las instancias es dinámica (**Eureka**).

Componente	Puerto	Responsabilidad
gateway	8443 HTTPS	Punto de entrada único: TLS, validación JWT/scope, enrutamiento, Retry, CircuitBreaker, X-Correlation-Id
auth-server	9000	Spring Authorization Server: emite JWT firmados con RSA; un cliente OAuth2 por canal
config-server / eureka-server	8888 / 8761	Configuración centralizada (config-repo) y descubrimiento de servicios
bff-web / bff-mobile / bff-atm	8084 / 8085 / 8086	Backend for Frontend por canal
customers-service	8087	Gestión de Clientes
accounts-service	8081	Gestión de Cuentas: apertura, cierre, mantenimiento, saldos
payments-service	8082	Procesamiento de Pagos: depósitos, retiros, pagos, transferencias
notification-service	8083	Consumidor de eventos: notificaciones y alertas de seguridad
batch-service	8090	Procesos batch migrados (Spring Batch)
PostgreSQL / Kafka	5432 / 9092	Una base por servicio (database-per-service) / mensajería de eventos

Decisiones de arquitectura

Decisión	Justificación
Un BFF por canal (3 módulos)	Permite desarrollo y despliegue en paralelo por equipo frontend y políticas distintas (ATM más estricto)
Gateway con TLS y validación de JWT + validación nuevamente en cada servicio	Defensa en profundidad: un acceso que evite el Gateway igual es rechazado
Database-per-service	Autonomía y aislamiento de fallas; la consistencia se asegura con idempotencia, SAGA y eventos
Kafka en lugar de JMS	Retención, particionado por cuenta (orden garantizado), reprocesamiento y escalado de consumidores
Reglas batch en Java puro (paquete domain)	Testeables sin Spring y verificables contra el legacy
Config Server perfil native	Simplicidad académica; en producción se apunta a Git/S3 cambiando una propiedad

5. Casos de uso

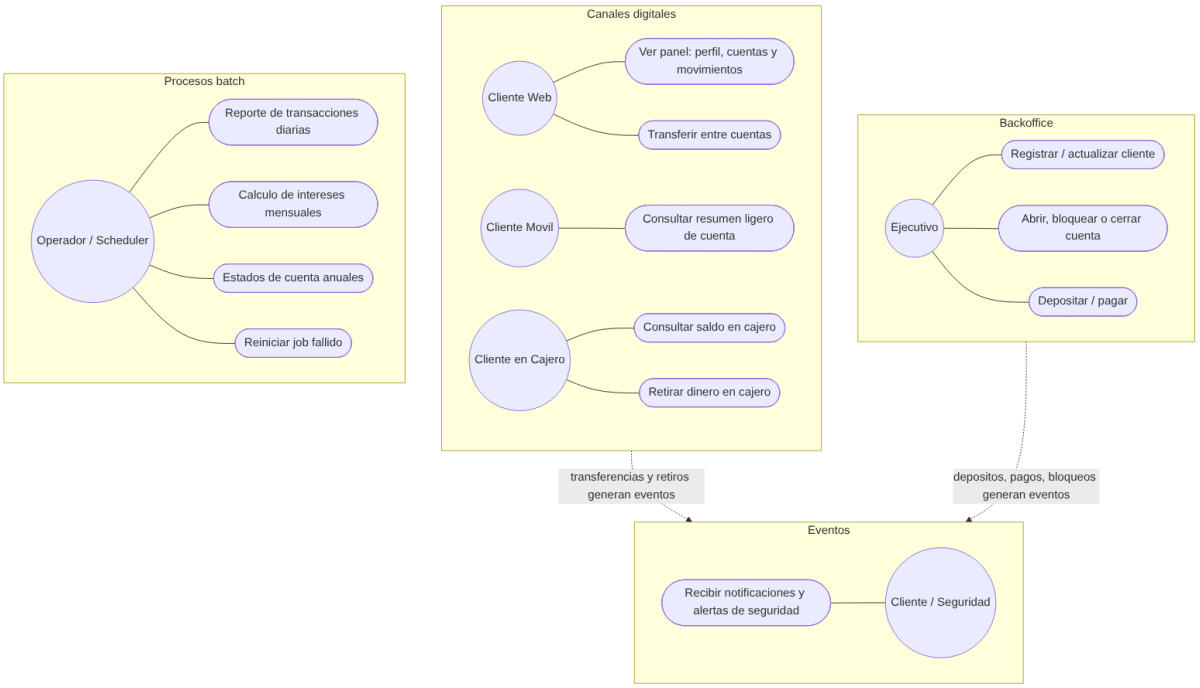


Figura 2. Diagrama de casos de uso

Los actores son el cliente en cada canal (web, móvil, cajero), el ejecutivo de backoffice, el operador/planificador batch y el propio cliente o el área de seguridad como receptores de notificaciones y alertas.

6. Parte 1 – Migración de procesos batch con Spring Batch



Figura 3. Jobs, steps y política ante fallos

Diseño de los jobs

Job (parámetro)	Steps	Salida
reporteTransaccionesDiariasJob (fechaProceso)	preparar → validar (chunk) → resumen por fecha (tasklet SQL) → exportar resumen → exportar anomalías	bt_transaccion_diaria, bt_resumen_diario, reporte_diario_FECHA.csv, anomalias_diarias_FECHA.csv
calculoInteresesMensualesJob (periodo)	preparar → cargar y deduplicar cuentas (chunk) → calcular intereses (**particionado**, 4 hilos) → exportar	bt_interes_mensual, intereses_mensuales_PERIODO.csv
estadosCuentaAnualesJob (anio)	cargar movimientos (chunk **multihilo**) → generar estados por cuenta (tasklet SQL) → exportar	bt_estado_cuenta_anual, estados_cuenta_ANIO.csv

Cada lectura usa `FlatFileItemReader` con un `LineMapper` propio que conserva el número de línea (trazabilidad). El `ItemProcessor` aplica las reglas de negocio y devuelve un resultado *válido* o *rechazado con motivo*; un `ClassifierCompositeItemWriter` envía cada uno a su tabla mediante `JdbcBatchItemWriter`. Ningún registro se pierde: los rechazados quedan en `bt_registro_rechazado`.

Reglas de calidad migradas

Archivo	Rechazos	Correcciones / alertas
movimientos_financieros_diarios	id inválido o duplicado, fecha inválida, tipo ≠ débito/crédito, monto vacío, negativo o cero	alerta MONTO_ALTO (> $2.500, configurable)
intereses_trimestrales	nombre vacío/"Unknown", saldo vacío o ≤ 0, edad fuera de 18–100, tipo inválido (-1, unknown), duplicado (se conserva el primer válido)	interés = saldo × tasa anual/12; ahorro 3 % (abono), préstamo 18 %, hipoteca 4,2 % (cargo)
estados_financieros_anuales	fecha inválida o fuera del año, tipo inválido, monto vacío, negativo o cero	"depósito" → "deposito"; descripción vacía → "Sin descripcion"

Se aceptan los cuatro formatos de fecha del legacy (`aaaa-mm-dd`, `dd-mm-aaaa`, `aaaa/mm/dd`, `dd/mm/aaaa`) con resolución estricta, que rechaza fechas imposibles como 31-02.

Manejo de excepciones, reintentos y reejecución

Mecanismo	Configuración
Reintento ante fallos temporales	`retry(TransientDataAccessException, DataAccessResourceFailureException)`, `retryLimit(3)`, backoff exponencial 0,5 s → 5 s
Omisión (skip)	Líneas corruptas (`FlatFileParseException`) se omiten hasta `skipLimit=1000` y se registran mediante un `SkipListener`
Reanudación	`JobRepository` en PostgreSQL; los readers guardan su posición: un job reiniciado continúa desde el último *commit*
Reejecución automática	`AutoRestartJobListener`: si un job termina FAILED se reinicia la misma JobInstance tras 20 s (máx. 2 intentos). No se reintenta si la causa es calidad de datos (skipLimit superado)
Idempotencia	Limpieza previa por clave de proceso y validación de duplicados reconstruida desde la BD: reejecutar produce siempre el mismo resultado
Políticas de finalización	`startLimit(5)`, validación de parámetros, `forzar=true` para reprocesar una instancia COMPLETED

Mecanismo	Configuración
Demostración controlada	Parámetros `simularFalloTransitorioEnLinea` y `simularFalloCriticoEnLinea`

Optimización para grandes volúmenes

- **Chunks de 200** registros con escritura JDBC por lotes (un viaje a la BD por chunk).
- **Particionamiento** del cálculo de intereses por rangos de `cuenta_id`, cada partición en su propio hilo y con su propio StepExecution (si falla una, solo se reprocesa esa).
- **Step multihilo** en estados anuales (`SynchronizedItemStreamReader` + pool de 4 hilos).
- Agregaciones en SQL (tasklets) en lugar de en memoria, e índices por clave de proceso.
- Parámetros ajustables sin recompilar en `config-repo/batch-service.yml` (chunk, hilos, particiones).

Integridad y equivalencia con el legacy

Como el código COBOL original no está disponible, la lógica legacy se reconstruyó en **Shell + awk** (`batch-service/legacy-reference`) a partir de las reglas de negocio, y se comparó con Spring Batch sobre los mismos archivos (semana_3, 1.000 registros cada uno). El redondeo `HALF_UP` reproduce la cláusula `ROUNDED` de COBOL. La comparación también se ejecuta en la prueba automatizada `EquivalenciaLegacyIntegrationTest`.

Indicador	Legacy (Shell/awk)	Spring Batch
Diario: válidos / alertas / días	392 / 65 / 239	392 / 65 / 239
Diario: total créditos / débitos	$244.500 / $284.900	$244.500 / $284.900
Diario: rechazos (tipo inválido / fecha / monto vacío / negativo / cero)	368 / 49 / 103 / 78 / 10	368 / 49 / 103 / 78 / 10
Intereses: cuentas válidas / duplicados	50 / 347	50 / 347
Intereses: ahorro / hipoteca / préstamo	$285,00 / $437,50 / $2.505,00	$285,00 / $437,50 / $2.505,00
Anual 2024: válidos / cuentas	604 / 20	604 / 20
Anual: depósitos / retiros / compras / pagos	$412.500 / $305.000 / $394.000 / $77.500	idéntico

7. Parte 2 – Patrón Backend for Frontend (BFF)

Aspecto	BFF Web	BFF Móvil	BFF Cajeros (ATM)
Endpoints	/web/customers/{id}/dashboard, /web/accounts/{id}/summary, POST /web/transfers	/mobile/accounts/{id}/summary, /mobile/customers/{id}/accounts	/atm/accounts/{id}/balance, POST /atm/withdrawals
Datos	Completos: perfil, todas las cuentas, historial, totales, advertencias	Esenciales: titular, saldo, 5 movimientos (fecha, tipo, monto); nulos omitidos	Mínimos: cuenta enmascarada (****0101), saldo; sin datos personales
Optimización	Una llamada del navegador reemplaza 3–4 llamadas	gzip, ETag (304 sin cuerpo), Cache-Control 15 s, límite en el origen	Timeouts y montos validados antes de llamar a pagos
Autorización	scope `web` (+ `bank.write` para operar)	scope `mobile` (solo lectura)	scope `atm` + terminal registrado `X-ATM-ID` + token de 5 min
Resiliencia	Retry 3 + CB + Bulkhead; respuesta degradada	1 reintento rápido; respuesta parcial	Sin datos viejos: 503; ante duda **no dispensa**

Aspecto	BFF Web	BFF Móvil	BFF Cajeros (ATM)
Reglas propias	Genera Idempotency-Key si falta	—	Idempotency-Key obligatoria; montos $5.000–$200.000 múltiplos de $5.000; auditoría por terminal

Comunicaciones seguras

- El Gateway atiende solo **HTTPS (TLS 1.2/1.3)** en el puerto 8443 con un certificado PKCS12 (en AWS, certificado de ACM en el balanceador).
- Los tokens se obtienen también por HTTPS a través del Gateway (`/oauth2/token`).
- Los JWT están firmados con RSA y se validan con las llaves públicas (JWKS) en el Gateway, en cada BFF y en cada microservicio. El BFF propaga el JWT original del canal.
- Los BFF son módulos independientes (código, imagen y despliegue propios): los equipos frontend y backend trabajan en paralelo.

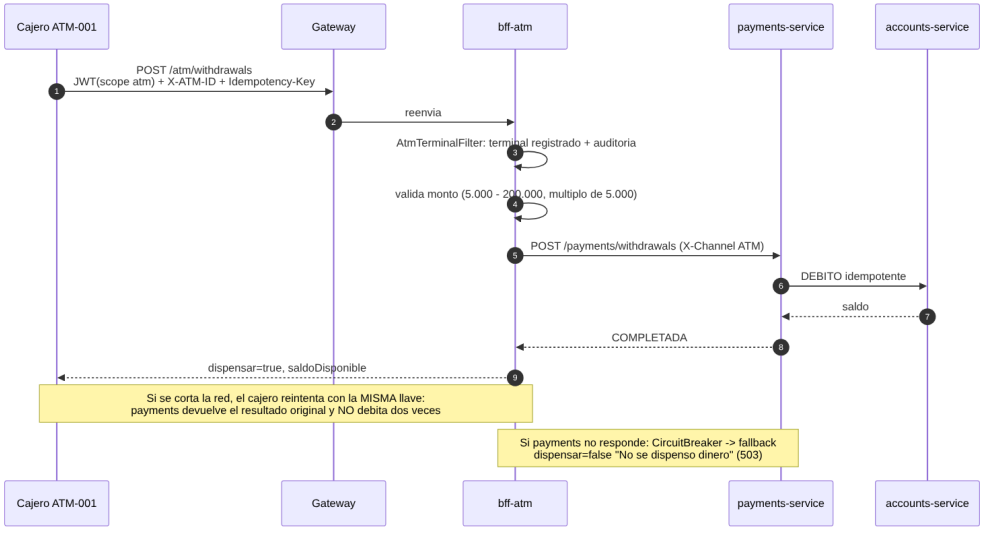


Figura 4. Secuencia de retiro en cajero con idempotencia

8. Parte 3 – Microservicios resilientes con Spring Cloud

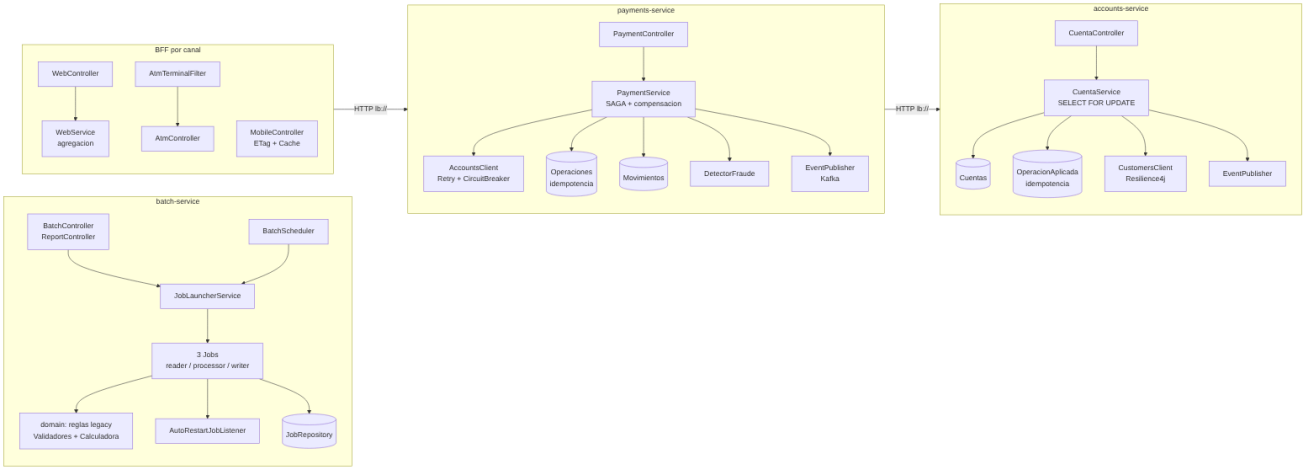


Figura 5. Diagrama de componentes principales

Microservicios de negocio

Servicio	Operaciones	Detalles relevantes
customers-service	GET/POST/PUT/DELETE /customers, /customers/rut/{rut}	Validación de RUT (módulo 11), baja lógica, bloqueo optimista @Version; alerta al cambiar email/teléfono
accounts-service	Apertura POST /accounts, mantenimiento PUT y PATCH /estado, cierre DELETE, POST /{id}/movements	Valida al titular en customers-service; cierre solo con saldo 0; débito/crédito con SELECT … FOR UPDATE e idempotente por operacionId
payments-service	POST /payments/deposits, /withdrawals, /bills, /transfers; GET /payments/account/{id}	Idempotency-Key; SAGA con compensación; detector de fraude (monto alto, retiros frecuentes)
notification-service	Consumidores Kafka; GET /notifications	Idempotente por eventId; 3 reintentos y Dead Letter Topic; métricas de alertas

Spring Cloud: configuración, descubrimiento, balanceo y enrutamiento

- **Config Server**: un archivo por servicio en `config-repo` más `application.yml` compartido; los valores se sobrescriben con variables de entorno (Docker/AWS).
- **Eureka**: todos los servicios se registran con un id de instancia único, lo que permite varias instancias del mismo servicio.
- **Balanceo**: Spring Cloud LoadBalancer (round robin) en el Gateway (`lb://`) y en los clientes `@LoadBalanced RestClient` .
- **Enrutamiento**: rutas por canal (`/web` , `/mobile` , `/atm`) y backoffice (`/api/**` con `StripPrefix`); filtro `Retry` para lecturas y `CircuitBreaker` con *fallback* por BFF.

Seguridad OAuth2

Cliente	Grant	Scopes
web-client	client_credentials	web, bank.read, bank.write
mobile-client	client_credentials	mobile, bank.read
atm-client	client_credentials (token 5 min)	atm, bank.read, bank.write
admin-client	client_credentials	web, mobile, atm, bank.read, bank.write, batch.admin
postman-client	authorization_code + refresh_token	todos (usuarios de prueba)

Lectura exige `bank.read` , escritura `bank.write` y la operación batch `batch.admin` , siguiendo el principio de mínimo privilegio.

Tolerancia a fallos (Resilience4j) y comportamiento alternativo

Llamada	Patrones	Comportamiento alternativo (fallback)
BFF web → servicios	Retry 3, CircuitBreaker (50 % de 10 llamadas, 15 s abierto), Bulkhead 20	Respuesta 200 con `degraded:true` y advertencias
BFF móvil → servicios	Retry 2 (200 ms), CircuitBreaker	Respuesta parcial no cacheable
BFF ATM → servicios	CircuitBreaker (sin datos viejos)	503 "Servicio temporalmente no disponible"; retiro: `dispensar=false`
payments → accounts	Retry 3 exponencial (seguro por idempotencia), CircuitBreaker con llamadas lentas	Operación RECHAZADA 503 "intente más tarde": nunca queda dinero a medias

Llamada	Patrones	Comportamiento alternativo (fallback)
accounts → customers	Retry 3, CircuitBreaker	No se abre la cuenta (503)
Gateway → BFF	CircuitBreaker + TimeLimiter 10 s	`/fallback/{bff}` 503 JSON

Los errores de negocio (4xx) se excluyen del Retry y del CircuitBreaker para no abrir el circuito por validaciones, y además se usan *timeouts* de conexión y lectura en todos los clientes HTTP.

Arquitectura orientada a eventos

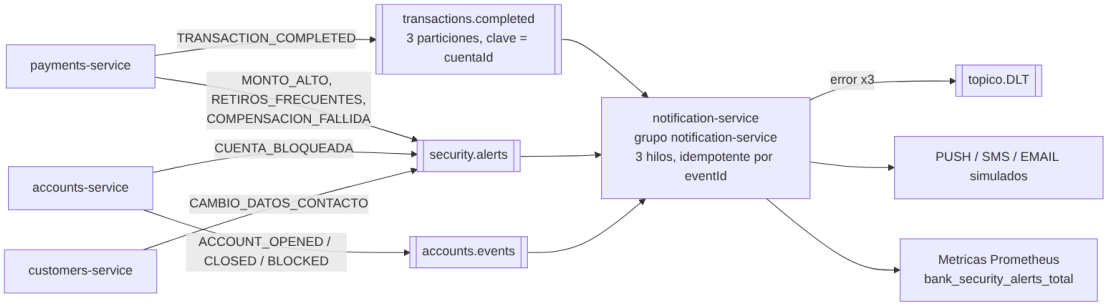


Figura 6. Tópicos, productores y consumidores

Tópico	Productores	Eventos
transactions.completed	payments-service	TRANSACTION_COMPLETED (depósito, retiro, pago, transferencia)
security.alerts	payments, accounts, customers	MONTO_ALTO, RETIROS_FRECUENTES, COMPENSACION_FALLIDA, CUENTA_BLOQUEADA, CAMBIO_DATOS_CONTACTO
accounts.events	accounts-service	ACCOUNT_OPENED / UPDATED / BLOCKED / ACTIVA / CLOSED
*.DLT	notification-service (recoverer)	Mensajes que fallan 3 veces

Los productores usan `acks=all` e idempotencia; la clave del mensaje es el id de cuenta (orden por cuenta). Hay 3 particiones por tópico y el consumidor usa 3 hilos dentro del grupo `notification-service`.

Consistencia de datos en un entorno distribuido

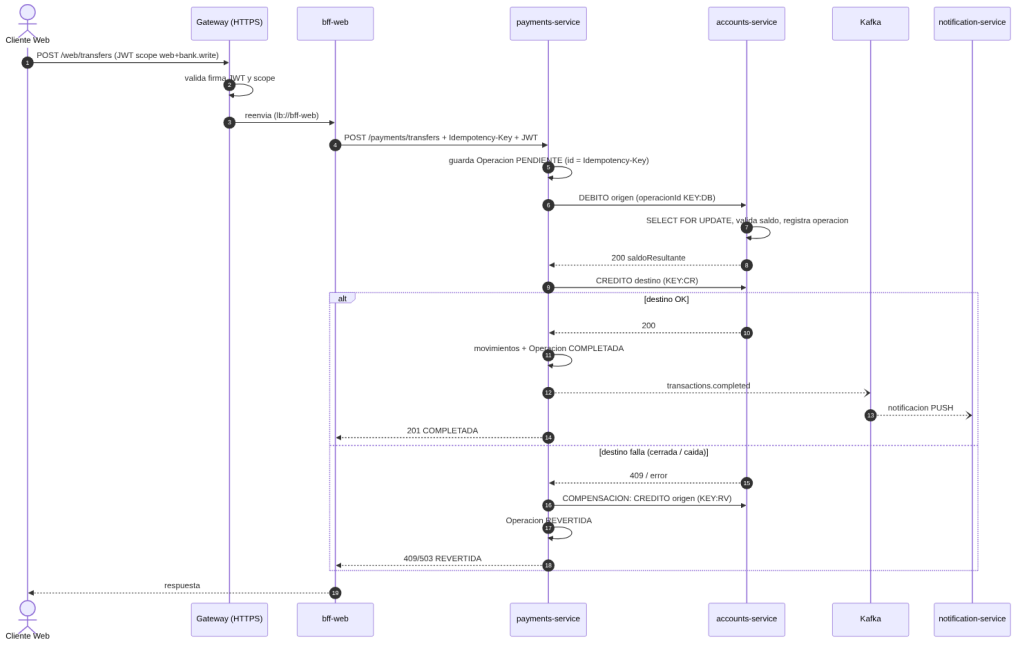


Figura 7. Transferencia con patrón SAGA y compensación

- **Atomicidad local**: el saldo se modifica con `SELECT … FOR UPDATE`; dos retiros simultáneos no pueden sobregirar la cuenta.
- **Idempotencia de extremo a extremo**: `Idempotency-Key` en pagos y `operacionId` en cuentas; los reintentos nunca duplican movimientos.
- **SAGA**: si falla el abono en la cuenta destino se compensa el cargo (estado REVERTIDA); si falla la compensación queda PENDIENTE_REVISION y se emite una alerta crítica.
- **Bloqueo optimista** (`@Version`) en clientes y cuentas para cambios administrativos concurrentes.

Monitoreo, registro y escalabilidad horizontal

- Actuator + Micrometer en todos los servicios (`/actuator/prometheus`, *probes* de liveness/readiness).
- Prometheus descubre automáticamente las instancias vía Eureka; Grafana incluye el dashboard *Banco XYZ* (tráfico, latencia p95, errores 5xx, circuit breakers, alertas, duración de jobs).
- Trazas distribuidas (Micrometer Tracing + Zipkin), con `traceId` en cada línea de log, también a través de Kafka; `X-Correlation-Id` y log de acceso en el Gateway; auditoría por terminal en el canal ATM.
- Servicios sin estado y sin puertos fijos: `docker compose up --scale payments-service=3`; en AWS, Auto Scaling de ECS por CPU.

9. Integración entre componentes

Un flujo de punta a punta ilustra cómo se integran los componentes:

1. El canal obtiene un token del **auth-server** a través del Gateway (HTTPS).
2. El **Gateway** valida el JWT y el scope, asigna un `X-Correlation-Id` y enruta al BFF usando Eureka y el balanceador.
3. El **BFF** valida nuevamente el token, consulta los microservicios necesarios (propagando el JWT) y protege cada llamada con Resilience4j.
4. Los **microservicios** aplican las reglas de negocio sobre su propia base de datos y publican eventos en **Kafka**.

5. El **notification-service** consume los eventos, genera notificaciones/alertas y expone métricas.
6. El **batch-service** procesa los archivos legacy y deja resultados en su base y en CSV, consultables por la API (`/api/batch/reports`).
7. Toda la configuración proviene del **Config Server** y las métricas y trazas llegan a Prometheus/Grafana/Zipkin.

10. Despliegue: contenedores y nube (AWS)

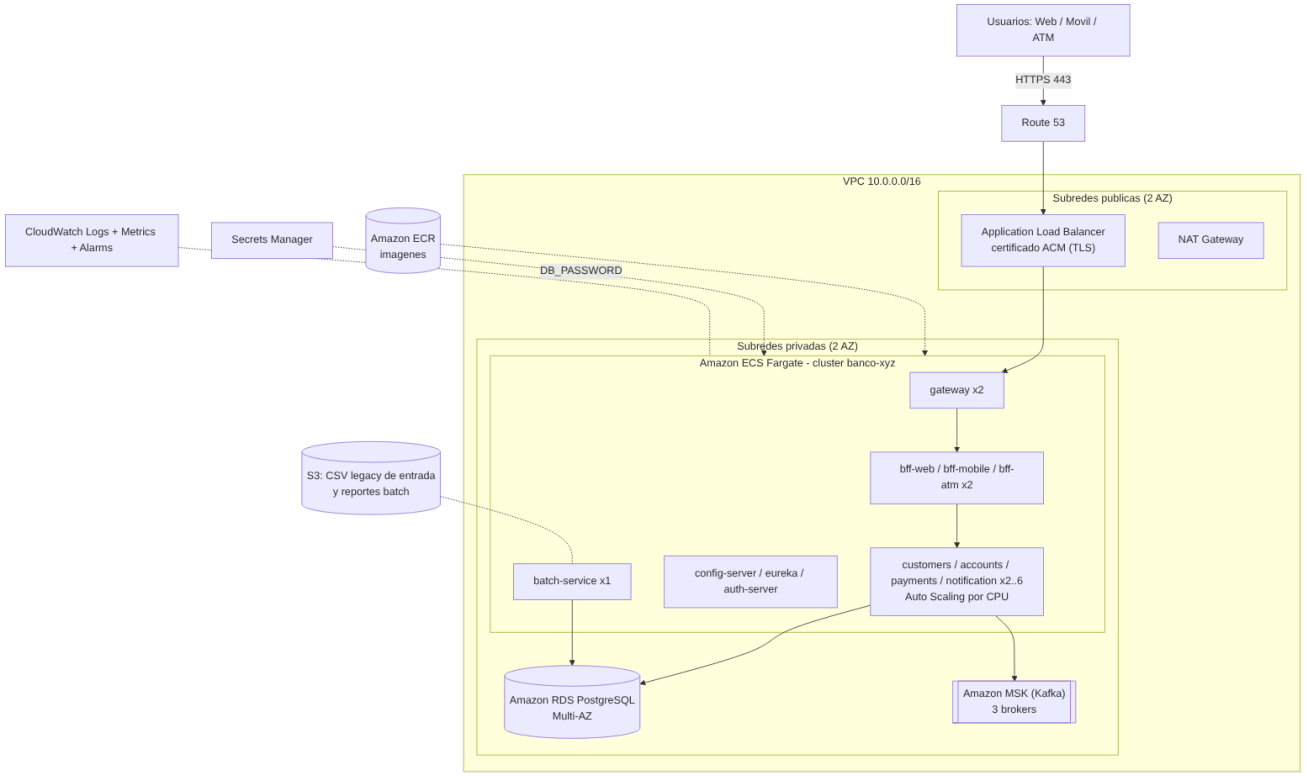


Figura 8. Arquitectura de despliegue objetivo en AWS

Local / EC2: `docker compose up --build -d` levanta 14 contenedores (perfil opcional `monitoring`). Las imágenes son multi-etapa, livianas y se ejecutan sin privilegios de root; solo el Gateway (8443) y las consolas de administración se publican.

AWS: se documentan dos opciones en `despliegue.md`: (A) una instancia EC2 con Docker Compose instalada mediante *user data*, y (B) producción con ECS Fargate, imágenes en ECR, RDS PostgreSQL Multi-AZ, Amazon MSK, ALB con certificado ACM, Secrets Manager, CloudWatch y Auto Scaling. Se incluyen los scripts `ec2-user-data.sh`, `ecr-push.sh`, la definición de tarea ECS y el override de Compose para servicios administrados.

11. Pruebas y validación

Tipo	Qué valida
Unitarias (ReglasDeNegocioTest, RutTest)	Formatos de fecha, normalización, detección de anomalías, duplicados, cálculo de intereses con redondeo, idempotencia ante reintentos, RUT
Integración batch (EquivalenciaLegacyIntegrationTest, H2)	Resultados idénticos al legacy en los 3 jobs; reinicio tras fallo crítico sin duplicar; recuperación de fallo transitorio; ejecución particionada
Pruebas funcionales (scripts/demo.sh, demo.ps1)	401/403 por canal, BFF web/móvil/ATM, idempotencia de retiros, clientes, cuentas, pagos, SAGA, eventos Kafka, batch y reinicio automático

Tipo	Qué valida
Resiliencia manual	Detención de accounts-service: respuestas degradadas, circuito OPEN → HALF_OPEN → CLOSED
Integración continua	GitHub Actions: `mvn verify`, scripts legacy y validación de docker-compose

12. Resultados y comparación con el sistema legacy

Dimensión	Legacy (COBOL + Shell, mainframe)	Nueva solución
Arquitectura	Monolito en mainframe	12 servicios independientes, una BD por servicio
Escalabilidad	Vertical, todo el sistema	Horizontal por servicio (réplicas, particiones Kafka, Auto Scaling)
Procesos batch	Scripts secuenciales; reejecución manual	Spring Batch con chunks, particiones, multihilo, reintentos y reinicio automático desde el último commit
Calidad de datos	Errores silenciosos	Cada rechazo registrado con motivo; reportes de anomalías
Canales	Mismo payload para todos	BFF por canal: completo, ligero (gzip/ETag) y seguro (ATM)
Seguridad	Centralizada, perimetral	OAuth2/JWT por canal, HTTPS, validación en cada servicio, mínimo privilegio
Tolerancia a fallos	Una falla afecta a todo	Fallas aisladas, circuit breakers y respuestas alternativas
Integración	Batch nocturno	Eventos en tiempo real (Kafka) y alertas de fraude
Observabilidad	Logs locales	Métricas, dashboards, trazas distribuidas y correlación
Despliegue	Manual en mainframe	Contenedores, Compose, CI y guía AWS

13. Desafíos enfrentados y soluciones

Desafío	Solución
Datos legacy con múltiples errores	Reglas aisladas en clases Java puras y probadas; registro de cada rechazo con motivo; STRICT en fechas
Demostrar equivalencia con el legacy	Reconstrucción del proceso en Shell/awk, comparación automatizada y redondeo HALF_UP = ROUNDED
Reintentar sin duplicar (batch y pagos)	Idempotencia: limpieza por clave, duplicados reconstruidos desde BD, Idempotency-Key y operacionId
Restart en steps multihilo (sin posición guardada)	Limpieza previa del step y reproceso completo; en steps secuenciales se reanuda desde el commit
Consistencia con bases separadas	SELECT FOR UPDATE, SAGA con compensación, eventos con acks=all, @Version
Fallbacks que no oculten errores de negocio	Excluir 4xx del Retry/CircuitBreaker y relanzarlos desde el fallback
Seguridad distinta por canal	Un cliente OAuth2 por canal, scopes en Gateway y BFF, segundo factor de terminal en ATM
Recursos limitados para 14 contenedores	Límites de memoria por contenedor, JVM ajustada (SerialGC, MaxRAMPercentage) y monitoreo en perfil opcional

14. Propuestas de mejora y próximos pasos

- Desplegar en AWS con ECS Fargate, RDS y MSK (scripts y guía incluidos) y agregar entrega continua desde GitHub Actions.
- Patrón **Transactional Outbox** para garantizar que todo cambio en la BD genere su evento.
- Persistir clientes OAuth2 y llaves RSA (KMS / Secrets Manager); **mTLS** entre servicios y TLS hacia RDS/MSK.
- Archivos batch en **S3**, programación con **EventBridge** y particionamiento remoto (varios nodos).
- Pruebas de contrato (Spring Cloud Contract), de carga (Gatling/k6) y de caos.
- Limitación de tasa (Redis) y AWS WAF en el borde; cache de lecturas frecuentes.

15. Conclusiones

La solución cumple los tres ejes de la evaluación. Los procesos batch se migraron a Spring Batch con resultados **verificablemente idénticos** al legacy y con capacidades que el legacy no tenía: reintentos, omisión controlada, procesamiento paralelo y reinicio automático. El patrón BFF entrega a cada canal exactamente lo que necesita, con seguridad propia y despliegue independiente. Los microservicios, integrados mediante Spring Cloud, OAuth2, Resilience4j y Kafka, aíslan fallas, mantienen la consistencia de los datos y permiten escalar horizontalmente. El sistema queda contenedorizado, observable y preparado para operar en AWS, sentando una base sólida para la evolución tecnológica del Banco XYZ.

16. Referencias

- Spring Batch Reference Documentation 5.1. https://docs.spring.io/spring-batch/reference/
- Spring Cloud (Config, Netflix Eureka, Gateway, CircuitBreaker) 2023.0. https://spring.io/projects/spring-cloud
- Spring Authorization Server y Spring Security OAuth2 Resource Server. https://docs.spring.io/spring-security/reference/
- Resilience4j Documentation. https://resilience4j.readme.io/
- Spring for Apache Kafka. https://docs.spring.io/spring-kafka/reference/
- Newman, S. (2021). *Building Microservices* (2.ª ed.). O'Reilly. Patrón Backend for Frontend.
- Richardson, C. (2018). *Microservices Patterns*. Manning. Patrones SAGA, idempotencia y Transactional Outbox.
- Amazon Web Services. Amazon ECS, RDS, MSK y Well-Architected Framework. https://docs.aws.amazon.com/
- Villagrán, K. fin_legacy_data. https://github.com/KariVillagran/fin_legacy_data